In [34]:

```python
import concurrent.futures
#import threading
import time

start = time.perf_counter()

def do_something(seconds):
    print (f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    print('Done sleeping...')

# t1 = threading.Thread(target = do_something, args = [2])
# #do_something()
# t1.start()
# t1.join()
# t2 = threading.Thread(target = do_something, args = [2])
# t2.start()
# t2.join()
threads = []
for _ in range(10):
    t1 = threading.Thread(target = do_something, args = [2])
    t1.start()
    threads.append(t1)
for i in threads:
    i.join()

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

```
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
sleeping 2 second (s)
Done sleeping...Done sleeping...

Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Finished in 2.01 second(s)
```

In [2]:

```python
import time

start = time.perf_counter()

def do_something():
    print('sleeping 1 second')
    time.sleep(1)
    print('Done sleeping...')

do_something()
do_something()

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

```
sleeping 1 second
Done sleeping...
sleeping 1 second
Done sleeping...
Finished in 2.0 second(s)
```

In [3]:
```python
import threading
import time

start = time.perf_counter()

def do_something():
    print('sleeping 1 second')
    time.sleep(1)
    print('Done sleeping...')

t1 = threading.Thread(target = do_something)
t2 = threading.Thread(target = do_something)

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

```
Finished in 0.0 second(s)
```

In [4]:
```python
import threading
import time

start = time.perf_counter()

def do_something():
    print('sleeping 1 second')
    time.sleep(1)
    print('Done sleeping...')

t1 = threading.Thread(target = do_something)
t2 = threading.Thread(target = do_something)

t1.start()
t2.start()

finish = time.perf_counter()
```

```
print(f'Finished in {round(finish-start,2)} second(s)')
```

```
sleeping 1 second
sleeping 1 second
Finished in 0.01 second(s)
Done sleeping...
Done sleeping...
```

In [6]:
```
import threading
import time

start = time.perf_counter()

def do_something():
    print('sleeping 1 second')
    time.sleep(1)
    print('Done sleeping...')

t1 = threading.Thread(target = do_something)
t2 = threading.Thread(target = do_something)

t1.start()
t2.start()

t1.join()
t2.join()

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

```
sleeping 1 second
sleeping 1 second
Done sleeping...
Done sleeping...
Finished in 1.01 second(s)
```

1. Creating 10 threads in a loop
2. Observe another loop to call the join() method
3. Why join() cannot be called after start() in the above loop? Why is the second loop required?

In [13]:
```
import threading
import time

start = time.perf_counter()

def do_something():
    print('sleeping 1 second')
    time.sleep(1)
    print('Done sleeping...')

threads = []

for _ in range(10):
    t = threading.Thread(target = do_something)
    t.start()
    threads.append(t)

for thread in threads:
    thread.join()


finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

1. Creating 10 threads inside a loop
2. Calling a function with argument
3. Remaining same as above

In [19]:
```
import threading
import time

start = time.perf_counter()

def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    print('Done sleeping...')
```

```
threads = []

for _ in range(10):
    t = threading.Thread(target = do_something, args = [1.5])
    t.start()
    threads.append(t)

for thread in threads:
    thread.join()

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

CONCURRENT FUTURES -1

1. Exploring ThreadPoolExecutor() in concurrent.futures
2. Using the context manager - with - to create an object for ThreadPoolExecutor() class
3. Pay attention to methods: executor.submit()..
4. f1 is a future object that stores the result returned by submit(), future means result of a task or a task waiting to run
5. the outcome can be accessed through result() method

In [31]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    return 'Done sleeping...'

with concurrent.futures.ThreadPoolExecutor() as executor:
    f1 = executor.submit(do_something, 1.5)
    f2 = executor.submit(do_something, 1.5)
    print(f1.result())
    print(f2.result())
    res=[executor.submit(do_something, 1.5) for _ in range(2)]
    # for _ in range(2):
    #     f1 = executor.submit(do_something, 1.5)
```

```
    for f in concurrent.futures.as_completed(res):
        print(f.result())

    # ffor _ in range(2):
finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

```
sleeping 1.5 second (s)
sleeping 1.5 second (s)
Done sleeping...
Done sleeping...
sleeping 1.5 second (s)
sleeping 1.5 second (s)
Done sleeping...
Done sleeping...
Finished in 3.02 second(s)
```

CONCURRENT FUTURES - 2

1. Creating many worker threads/ Thread pool using loop/list comprehension
2. understanding as_completed(results) - returns an iterator object which fetches the result as and when it is completed

In [25]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    return 'Done sleeping...'

with concurrent.futures.ThreadPoolExecutor() as executor:
    results = [executor.submit(do_something, 1) for _ in range(10)]

    for f in concurrent.futures.as_completed(results):
        print(f.result())

finish = time.perf_counter()
```

```
print(f'Finished in {round(finish-start,2)} second(s)')
```

CONCURRENT FUTURES - 3

1. List comprehension + as_completed + list of arguments to the function secs = [5, 4, 3, 2 , 1]
2. change the function to return the seconds to understand the order of execution

In [ ]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    return f'Done sleeping...'

with concurrent.futures.ThreadPoolExecutor() as executor:
    results = [executor.submit(do_something, 1) for _ in range(10)]

    for f in concurrent.futures.as_completed(results):
        print(f.result())

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

In [27]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    return f'Done sleeping {seconds}...'

with concurrent.futures.ThreadPoolExecutor() as executor:
    secs = [5, 4, 3, 2, 1]
```

```
    results = [executor.submit(do_something, sec) for sec in secs]

    for f in concurrent.futures.as_completed(results):
        print(f.result())

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

CONCURRENT FUTURES - 4

Using the built-in map() method

Difference: map does not hold a future object (only submit does that), it holds the results in the order they were started

In [32]:
```
import concurrent.futures
import time

start = time.perf_counter()


def do_something(seconds):
    print(f'Sleeping {seconds} second(s)...')
    time.sleep(seconds)
    return f'Done Sleeping...{seconds}'

with concurrent.futures.ThreadPoolExecutor() as executor:
    secs = [5, 4, 3, 2, 1]
    results = executor.map(do_something, secs)

for result in results:
    print(result)

finish = time.perf_counter()

print(f'Finished in {round(finish-start, 2)} second(s)')
```

```
Sleeping 5 second(s)...
Sleeping 4 second(s)...
Sleeping 3 second(s)...
Sleeping 2 second(s)...
Sleeping 1 second(s)...
Done Sleeping...5
Done Sleeping...4
Done Sleeping...3
Done Sleeping...2
Done Sleeping...1
Finished in 5.01 second(s)
```

executor → manages threads

submit() → submits a task

Future → represents the task/result

f1 → variable holding a Future

f1.result() → gets the actual result

results → list containing multiple Futures

We will see a simple IO task - downloading high resolution images from a website

In [33]:

```python
import time
import requests

img_urls = [
    'https://unsplash.com/photos/raindrops-on-window-with-palm-silhouette-txzbJfDbC0I',
    'https://unsplash.com/photos/forested-hillside-overlooking-calm-sea-x-wmJ_IurGw',
    'https://unsplash.com/photos/stone-alleyway-with-fabric-canopies-cCmj0YhF9Do',
    'https://unsplash.com/photos/pink-peony-flower-on-green-stem-q1WtkPjHmJM',
    'https://unsplash.com/photos/brown-suede-loafers-and-buttoned-garment-R3_FVgwAEv4',
    'https://unsplash.com/photos/star-trails-over-mountain-peak-amrQhXZWtYE',
    'https://unsplash.com/photos/sunflowers-in-field-at-golden-hour-01k_oAMpt7E',
    'https://unsplash.com/photos/shooting-star-over-dark-treeline-and-house-UrGB7kftxwk',
    'https://unsplash.com/photos/fortified-church-in-biertan-romania-ZdEuB3lxTfg',
    'https://unsplash.com/photos/sunflower-in-golden-hour-light-xRkLHDRWPac'
]
```

```
t1 = time.perf_counter()

for img_url in img_urls:
    img_bytes = requests.get(img_url).content
    img_name = img_url.split('/')[4]
    img_name = f'{img_name}.jpg'
    with open(img_name, 'wb') as img_file:
        img_file.write(img_bytes)
        print(f'{img_name} was downloaded...')

t2 = time.perf_counter()
print(f'Finished in {t2-t1} seconds')
```

```
raindrops-on-window-with-palm-silhouette-txzbJfDbC0I.jpg was downloaded...
forested-hillside-overlooking-calm-sea-x-wmJ_IurGw.jpg was downloaded...
stone-alleyway-with-fabric-canopies-cCmj0YhF9Do.jpg was downloaded...
pink-peony-flower-on-green-stem-q1WtkPjHmJM.jpg was downloaded...
brown-suede-loafers-and-buttoned-garment-R3_FVgwAEv4.jpg was downloaded...
star-trails-over-mountain-peak-amrQhXZWtYE.jpg was downloaded...
sunflowers-in-field-at-golden-hour-01k_oAMpt7E.jpg was downloaded...
shooting-star-over-dark-treeline-and-house-UrGB7kftxwk.jpg was downloaded...
fortified-church-in-biertan-romania-ZdEuB3lxTfg.jpg was downloaded...
sunflower-in-golden-hour-light-xRkLHDRWPac.jpg was downloaded...
Finished in 1.4669260999653488 seconds
```

The above code is a good candidate for Threads. While one thread waits for a download, the next one can start the download. Each image can be of different sizes. Smallest ones if started can download faster.

To do this:

1. We will create a function - argument for executor
2. import concurrent.futures
3. After the function, create the ThreadPoolExecutor() in a context manager
4. As it is an IO bound task, Threads are effective - downloading high resolution images, observe that the time is took was far lesser compared to the earlier version
5. Threads will not be effective if processing of images like resizing or feature extraction are involved

In [34]:
```
import time
import requests
```

```python
import concurrent.futures


img_urls = [
    'https://unsplash.com/photos/raindrops-on-window-with-palm-silhouette-txzbJfDbC0I',
    'https://unsplash.com/photos/forested-hillside-overlooking-calm-sea-x-wmJ_IurGw',
    'https://unsplash.com/photos/stone-alleyway-with-fabric-canopies-cCmj0YhF9Do',
    'https://unsplash.com/photos/pink-peony-flower-on-green-stem-q1WtkPjHmJM',
    'https://unsplash.com/photos/brown-suede-loafers-and-buttoned-garment-R3_FVgwAEv4',
    'https://unsplash.com/photos/star-trails-over-mountain-peak-amrQhXZWtYE',
    'https://unsplash.com/photos/sunflowers-in-field-at-golden-hour-01k_oAMpt7E',
    'https://unsplash.com/photos/shooting-star-over-dark-treeline-and-house-UrGB7kftxwk',
    'https://unsplash.com/photos/fortified-church-in-biertan-romania-ZdEuB3lxTfg',
    'https://unsplash.com/photos/sunflower-in-golden-hour-light-xRkLHDRWPac'
]

t1 = time.perf_counter()

def download_urls (img_url):
    img_bytes = requests.get(img_url).content
    img_name = img_url.split('/')[4]
    img_name = f'{img_name}.jpg'
    with open(img_name, 'wb') as img_file:
        img_file.write(img_bytes)
        print(f'{img_name} was downloaded...')

with concurrent.futures.ThreadPoolExecutor() as executor:
    executor.map(download_urls, img_urls)

t2 = time.perf_counter()
print(f'Finished in {t2-t1} seconds')
```

```
forested-hillside-overlooking-calm-sea-x-wmJ_IurGw.jpg was downloaded...
star-trails-over-mountain-peak-amrQhXZWtYE.jpg was downloaded...
sunflowers-in-field-at-golden-hour-01k_oAMpt7E.jpg was downloaded...
shooting-star-over-dark-treeline-and-house-UrGB7kftxwk.jpg was downloaded...
stone-alleyway-with-fabric-canopies-cCmj0YhF9Do.jpg was downloaded...
sunflower-in-golden-hour-light-xRkLHDRWPac.jpg was downloaded...
fortified-church-in-biertan-romania-ZdEuB3lxTfg.jpg was downloaded...
raindrops-on-window-with-palm-silhouette-txzbJfDbC0I.jpg was downloaded...
brown-suede-loafers-and-buttoned-garment-R3_FVgwAEv4.jpg was downloaded...
pink-peony-flower-on-green-stem-q1WtkPjHmJM.jpg was downloaded...
Finished in 1.9803520999848843 seconds
```

In [7]:

```python
import queue
import threading
import time
thread_exit_Flag = 0
class sample_Thread (threading.Thread):
   def __init__(self, threadID, name, q):
      threading.Thread.__init__(self)
      self.threadID = threadID
      self.name = name
      self.q = q
   def run(self):
      print ("initializing " + self.name)
      process_data(self.name, self.q)
      print ("Exiting " + self.name)
# helper function to process data
def process_data(threadName, q):
   while not thread_exit_Flag:
      queueLock.acquire()
      if not workQueue.empty():
         data = q.get()
         queueLock.release()
         print ("% s processing % s" % (threadName, data))
      else:
         queueLock.release()
         time.sleep(1)
thread_list = ["Thread-1", "Thread-2", "Thread-3"]
name_list = ["A", "B", "C", "D", "E"]
queueLock = threading.Lock()
workQueue = queue.Queue(10)
threads = []
```

```
threadID = 1
# Create new threads
for thread_name in thread_list:
   thread = sample_Thread(threadID, thread_name, workQueue)
   thread.start()
   threads.append(thread)
   threadID += 1
# Fill the queue
queueLock.acquire()
for items in name_list:
   workQueue.put(items)
queueLock.release()
# Wait for the queue to empty
while not workQueue.empty():
   pass
# Notify threads it's time to exit
thread_exit_Flag = 1
# Wait for all threads to complete
for t in threads:
   t.join()
print ("Exit Main Thread")
```

```
initializing Thread-1
initializing Thread-2
initializing Thread-3
Thread-2 processing A
Thread-3 processing B
Thread-1 processing C
Thread-1 processing D
Thread-1 processing E
Exiting Thread-1
Exiting Thread-3
Exiting Thread-2
Exit Main Thread
```

In [4]:

```
import threading

counter = 0

def increment_counter():
    global counter
    for _ in range(100000):
```

```
        #time.sleep(.0001)
        counter +=1

t1 = threading.Thread(target = increment_counter)
t2 = threading.Thread(target = increment_counter)

t1.start()
t2.start()

t1.join()
t2.join()

print(counter)
```

```
200000
```

In [10]:

```
import threading
import time

counter = 0

def increment_counter():
    global counter

    for _ in range(1000):
        temp = counter       # Read shared value

        #time.sleep(0.0001)   # Give another thread a chance to run

        counter = temp + 1   # Write shared value


t1 = threading.Thread(target=increment_counter)
t2 = threading.Thread(target=increment_counter)

t1.start()
t2.start()

t1.join()
t2.join()
```

```
print("Expected:", 2000)
print("Actual:", counter)
```

```
Expected: 2000
Actual: 2000
```

1. Practice the above Multithreading snippets using both conventional and advanced modules
2. Appreciate the use of Mutltithreading Priority Queue to facilitate resource access for multiple threads|
3. Design a Multithreading Application that downloads ten high resolution images with a note on the time taken
4. Report on the errors and how you fixed them with screenshots
5. After local download use two threads to access and display the images without redundancy - use a priority queue
6. Make a crisp comment on the following:
    - Are queueLock and workQueue automatically connected? How? explain with respect to their working
    - Is the main thread loading the queue first for the worker threads to access the queue items? Explain

In [ ]: